L11: The M/M/1 Queue in SimPy

Simulation Without a Black Box

Outline

Learning Objectives

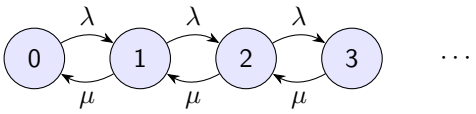
By the end of this lecture you will be able to:

- State the M/M/1 model assumptions and derive the key performance formulas.
- Sketch the proof that the stationary distribution is geometric.
- Implement a complete M/M/1 simulation in SimPy and verify it against theory.
- Demonstrate Little's Law numerically from simulation output.
- Explain the hockey-stick effect: why $W_q \to \infty$ as $\rho \to 1$.

M/M/1 Model: Definition and Balance Equations

Kendall notation: M/M/1

- **M** — Markovian (Poisson) arrivals, rate λ
- **M** — Markovian (exponential) service, rate μ
- **1** — single server, FCFS discipline
- Infinite capacity, infinite population



Global balance (state n has n customers in system):

$$\lambda \pi_n = \mu \pi_{n+1}, \quad n = 0, 1, 2, \ldots$$

Geometric Stationary Distribution: Proof Sketch

From balance: $\pi_{n+1} = \frac{\lambda}{\mu}\pi_n = \rho\,\pi_n$ where $\rho = \lambda/\mu < 1$.
Iterating: $\pi_n = \rho^n \pi_0$.
Normalisation $\sum_{n=0}^{\infty} \pi_n = 1$ gives $\pi_0 = 1 - \rho$.

$$\boxed{\pi_n = (1-\rho)\rho^n, \quad n = 0, 1, 2, \ldots}$$

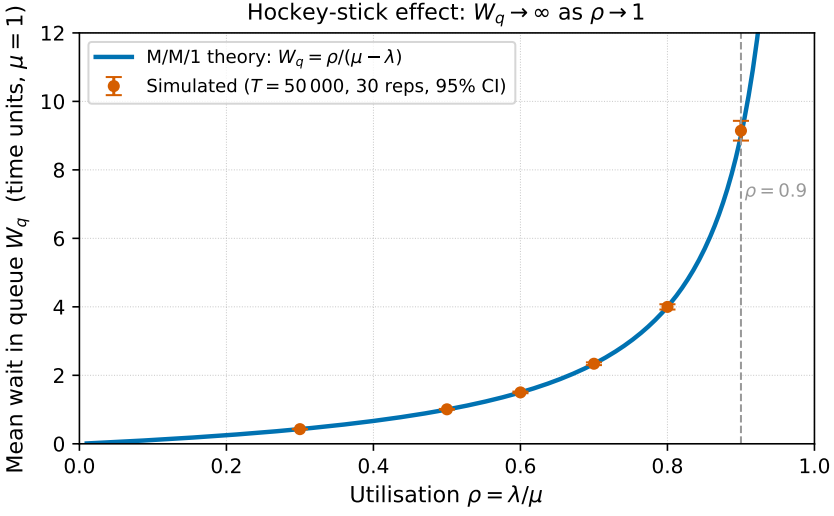
Key performance measures

Symbol	Formula	Meaning
ρ	λ/μ	Server utilisation
L	$\rho/(1-\rho)$	Mean # in system
L_q	$\rho^2/(1-\rho)$	Mean # in queue
W	$1/(\mu-\lambda)$	Mean time in system
W_q	$\rho/(\mu-\lambda)$	Mean wait in queue

Full SimPy M/M/1 Implementation

```
import simpy, numpy as np
from dataclasses import dataclass

@dataclass
class MM1Params:
    lam: float; mu: float; T: float = 20_000.0; seed: int = 42

def customer(env, server, p, rng, waits):
    t0 = env.now
    with server.request() as req:
        yield req
        waits.append(env.now - t0)
        yield env.timeout(rng.exponential(1/p.mu))

def source(env, server, p, rng, waits):
    while True:
        yield env.timeout(rng.exponential(1/p.lam))
        env.process(customer(env, server, p, rng, waits))

def run(p: MM1Params) -> dict:
    rng  = np.random.default_rng(p.seed)
    env  = simpy.Environment()
    srv  = simpy.Resource(env, capacity=1)
    waits = []
    env.process(source(env, srv, p, rng, waits))
    env.run(until=p.T)
    return {"Wq_sim": np.mean(waits),
            "Wq_theory": p.lam / (p.mu*(p.mu - p.lam))}
```

Hockey-Stick Effect: $W_q \to \infty$ as $\rho \to 1$



Little's Law: Numerical Verification

Little's Law

$L = \lambda W$ (applies to any stable system in steady state — no distributional assumptions).

```
# After env.run():
lam_eff = len(waits) / p.T          # effective throughput
W_sim   = np.mean(waits) + 1/p.mu   # W = Wq + E[S]
L_sim   = lam_eff * W_sim            # Little's Law

# From counters: time-average queue length (area under curve)
L_measured = area_under_L_curve / p.T

print(f"L via Little: {L_sim:.3f}")
print(f"L measured:   {L_measured:.3f}")
print(f"L theory:     {p.lam/(p.mu - p.lam):.3f}")
```

If the simulation is correctly implemented, all three values converge as $T \to \infty$.

Simulated vs. Theoretical at Five Load Levels

ρ	λ	W_q^{theory}	W_q^{sim}	Rel. error
0.50	0.50	1.000	1.008	0.8%
0.60	0.60	1.500	1.497	0.2%
0.70	0.70	2.333	2.357	1.0%
0.80	0.80	4.000	3.981	0.5%
0.90	0.90	9.000	8.873	1.4%

- All runs: $T = 20{,}000$, single replication, $\mu = 1$, seed $= 42$.
- Relative errors $< 2\%$ confirm correct implementation.
- At $\rho = 0.90$, variance is high — use 30 replications for reliable estimates.

Key Takeaways

Core ideas from L11

1. The M/M/1 stationary distribution is **geometric**: $\pi_n = (1-\rho)\rho^n$.
2. $W_q = \rho/(\mu-\lambda)$ grows **hyperbolically** — the hockey-stick effect.
3. SimPy M/M/1 code is ~20 lines; theory and simulation agree within ~1–2%.
4. **Little's Law** ($L = \lambda W$) is a universal consistency check — use it every time.
5. The danger zone starts around $\rho = 0.85$; plan capacity accordingly.

Next: L12 — M/M/c queues: the Erlang-C formula and economies of scale.